

 bclavie / **JaCo1BERTv2**   like 16

 Sentence Similarity

 Safetensors

 bclavie/mmarco-japanese-hard-negatives

 uncamp-dl/mmarco

 Japanese

 RAGatouille

 bert

 ColBERT

 arxiv:2312.16144

 arxiv:2310.19349

 arxiv:2112.01488

 License: mit



 **Model card**  Files  xet  Community **2**

Downloads last month
18,475



 **Safetensors** 

Model size 111M params Tensor type F32  Files info

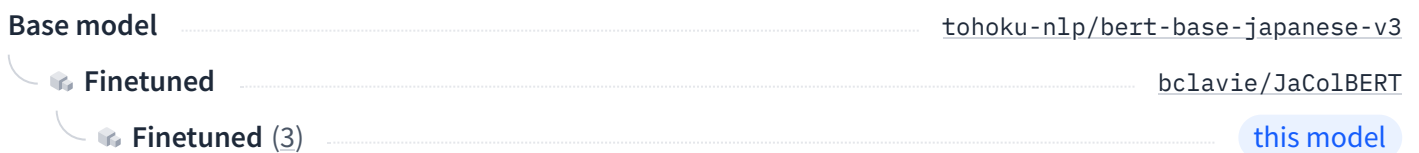
 **Inference Providers** **NEW**

 Sentence Similarity

This model isn't deployed by any Inference Provider.

 Ask for provider support

 **Model tree for bclavie/JaCo1BERTv2**



 **Datasets used to train bclavie/JaCo1BERTv2**

 **Collection including bclavie/JaCo1BERTv2**

JaCo1BERT  Collection

7 items • Updated Aug 16, 2024 •  1

First version of JaCo1BERTv2. Weights might be updated in the next few days.

Current early checkpoint is fully functional and outperforms multilingual-e5-large, BGE-M3 and JaColBERT in early results, but full evaluation TBD.# Intro

“There is currently no JaColBERTv2 technical report. For an overall idea, you can refer to the JaColBERTv1 [arXiv Report](#)”

If you just want to check out how to use the model, please check out the [Usage section](#) below!

Welcome to JaColBERT version 2, the second release of JaColBERT, a Japanese-only document retrieval model based on [ColBERT](#).

JaColBERTv2 is a model that offers very strong out-of-domain generalisation. Having been only trained on a single dataset (MMarco), it reaches state-of-the-art performance.

JaColBERTv2 was initialised off JaColBERTv1 and trained using knowledge distillation with 31 negative examples per positive example. It was trained for 250k steps using a batch size of 32.

The information on this model card is minimal and intends to give a quick overview! It'll be updated once benchmarking is complete and a longer report is available.

🔗 Why use a ColBERT-like approach for your RAG application?

Most retrieval methods have strong tradeoffs:

- **Traditional sparse approaches**, such as BM25, are strong baselines, **but** do not leverage any semantic understanding, and thus hit a hard ceiling.
- **Cross-encoder** retriever methods are powerful, **but** prohibitively expensive over large datasets: they must process the query against every single known document to be able to output scores.
- **Dense retrieval** methods, using dense embeddings in vector databases, are lightweight and perform well, **but** are **not** data-efficient (they often require hundreds of millions if not billions of training examples pairs to reach state-of-the-art performance) and generalise poorly in a lot of cases. This makes sense: representing every single aspect of a document, to be able to match it to any potential query, into a single vector is an extremely hard problem.

ColBERT and its variants, including JaColBERTv2, aim to combine the best of all worlds: by representing the documents as essentially *bags-of-embeddings*, we obtain superior performance and strong out-of-domain generalisation at much lower compute cost than cross-encoders.

🔗 Training

🔗 Training Data

The model is trained on the japanese split of MMARCO. It uses ColBERTv2 style training, meaning the model uses knowledge distillation from a cross-encoder model. We use the same cross-encoder scores as the original English ColBERTv2 training (as MMarco is a translated dataset, these are more or less well mapped). These scores are available [here](#).

Unlike English ColBERTv2, we use nway=32 rather than nway=64, meaning that we provide the model with 31 negative examples per positive examples. Furthermore, we downsample the original sets of triplets from over 19 million to 8 million examples.

🔗 Training Method

JColBERT is trained for a single epoch (1-pass over every triplet, meaning 250000 trainings steps) on 8 NVidia A100 40GB GPUs. Total training time was around 30 hours.

JColBERT is initialised from [JaColBERT](#), which itselfs builds upon Tohoku University's excellent [bert-base-japanese-v3](#). Our experiments benefitted strongly from Nagoya University's work on building [strong Japanese SimCSE models](#), among other work.

JaColBERT is trained with an overall batch size of 32 and a learning rate of 1e-5, and a warmup of 20000 steps. Limited exploration was performed but those defaults outperformed other experiments.

JaColBERT, as mentioned above, uses knowledge distillation using cross-encoder scores generated by a MiniLM cross-encoder on the English version of MS Marco. Please refer to the original [ColBERTv2 paper](#) for more information on this approach.

🔗 Results

We present the first results, on two datasets: JQaRa, a passage retrieval task composed of questions and wikipedia passages containing the answer, and JSQuAD, the Japanese translation of SQuAD. (Further evaluations on MIRACL and TyDi are running, but fairly slow due to how long it takes to run e5-large and bge-m3.)

JaColBERTv2 reaches state-of-the-art results on both datasets, outperforming models with 5x more parameters.

		JQaRa				JSQuAD		
	NDCG@10	MRR@10	NDCG@100	MRR@100		R@1	R@5	R@10
JaColBERTv2	0.585	0.836	0.753	0.838		0.921	0.977	0.982
JaColBERT	0.549	0.811	0.730	0.814		0.913	0.972	0.978
bge-m3+all	0.576	0.818	0.745	0.820		N/A	N/A	N/A
bg3-m3+dense	0.539	0.785	0.721	0.788		0.850	0.959	0.976
m-e5-large	0.554	0.799	0.731	0.801		0.865	0.966	0.977
m-e5-base	0.471	0.727	0.673	0.731		0.838	0.955	0.973
m-e5-small	0.492	0.729	0.689	0.733		0.840	0.954	0.973
GLuCoSE	0.308	0.518	0.564	0.527		0.645	0.846	0.897
sup-simcse-ja-base	0.324	0.541	0.572	0.550		0.632	0.849	0.897
sup-simcse-ja-large	0.356	0.575	0.596	0.583		0.603	0.833	0.889
fio-base-v0.1	0.372	0.616	0.608	0.622		0.700	0.879	0.924

[🔗 Usage](#)

[🔗 Installation](#)

JaColBERT works using ColBERT+RAGatouille. You can install it and all its necessary dependencies by running:

```
pip install -U ragatouille
```

For further examples on how to use RAGatouille with ColBERT models, you can check out the [examples section](#) [in the github repository](#).

Specifically, example 01 shows how to build/query an index, 04 shows how you can use JaColBERTv2 as a re-ranker, and 06 shows how to use JaColBERTv2 for in-memory searching rather than using an index.

Notably, RAGatouille has metadata support, so check the examples out if it's something you need!

🔗 Encoding and querying documents without an index

If you want to use JaColBERTv2 without building an index, it's very simple, you just need to load the model, `encode()` some documents, and then `search_encoded_docs()`:

```
from ragatouille import RAGPretrainedModel
RAG = RAGPretrainedModel.from_pretrained("bclavie/JaColBERTv2")

RAG.encode(['document_1', 'document_2', ...])
RAG.search_encoded_docs(query="your search query")
```

Subsequent calls to `encode()` will add to the existing in-memory collection. If you want to empty it, simply run `RAG.clear_encoded_docs()`.

🔗 Indexing

In order for the late-interaction retrieval approach used by ColBERT to work, you must first build your index. Think of it like using an embedding model, like e5, to embed all your documents and storing them in a vector database. Indexing is the slowest step retrieval is extremely quick. There are some tricks to speed it up, but the default settings work fairly well:

```
from ragatouille import RAGPretrainedModel

RAG = RAGPretrainedModel.from_pretrained("bclavie/JaColBERT")
documents = [ "マクドナルドのフライドポテトの少量のカロリーはいくつですか？マクドナルドの小さな
RAG.index(name="My_first_index", collection=documents)
```

The index files are stored, by default, at `.ragatouille/colbert/indexes/{index_name}`.

And that's it! Let it run, and your index and all its representations (compressed to 2bits by default) will have been generated.

🔗 Searching

Once you have created an index, searching through it is just as simple! If you're in the same session and `RAG` is still loaded, you can directly search the newly created index. Otherwise, you'll want to load it from disk:

```
RAG = RAGPretrainedModel.from_index(".ragatouille/colbert/indexes/My_first_index")
```

And then query it:

```
RAG.search(query="QUERY")
> [{ 'content': 'TEXT OF DOCUMENT ONE',
      'score': float,
      'rank': 1,
      'document_id': str,
      'document_metadata': dict},
    { 'content': 'TEXT OF DOCUMENT TWO',
      'score': float,
      'rank': 2,
      'document_id': str,
      'document_metadata': dict},
    [...]
  ]
```

Citation

If you'd like to cite this work, please cite the JaColBERT technical report:

```
@misc{clavié2023jacolbert,  
  title={JaColBERT and Hard Negatives, Towards Better Japanese-First Embedding},  
  author={Benjamin Clavié},  
  year={2023},  
  eprint={2312.16144},  
  archivePrefix={arXiv},  
  primaryClass={cs.CL}  
}
```